PROYECTO FINAL
BASES DE DATOS

LOLA MANCEBO OLIVARES Y JIMENA ZUBIZARRETA GÓMEZ

Contenido

1. Introducción 3
2. Diseño y carga de datos 4
   2.1 Diseño del modelo relacional (MySQL) 4
   2.2 Diseño del modelo no relacional (MongoDB) 5
   2.3 Proceso de carga de datos 6
   2.4 Archivo de configuración 6
3. Aplicación Python para el acceso y visualización de los datos 7
   3.1 Estructura general de la aplicación 7
   3.2 Tecnologías utilizadas 7
   3.3 Evolución de reviews por año 8
   3.4 Evolución de la popularidad de los artículos 8
   3.5 Histograma por nota 9
   3.6 Evolución de las reviews a lo largo del tiempo 10
   3.7 Histograma de reviews por usuario 10
   3.8 Nube de palabras 11
   3.9 Gráfica adicional: media de puntuaciones por artículo 12
   3.10 Conclusión 12
4. Aplicación Python junto con Neo4J 13
   4.1 Obtener similitudes entre usuarios y mostrar los enlaces en Neo4J 13
      Comprobaciones del código con consultas Neo4J 14
   4.2 Obtener enlaces entre usuarios y artículos 15
      Comprobaciones del código con consultas Neo4J 16
   4.3 Obtener algunos usuarios que han visto más de un determinado tipo de artículo 19
      Comprobaciones del código con consultas Neo4J 20
   4.4 Artículos populares y artículos en común entre usuarios 21
      Comprobaciones del código con consultas Neo4J 22
   4.5 Conclusión 23
5. Nuevos datos 25
6. Más visualización y relación con Machine Learning 27
   6.1 Visualización con herramienta externa 27
      Evolución de reviews por año 27
      Evolución de la popularidad de los artículos 28
      Nube de palabras 29

6.2 Diseño de un modelo de recomendación.....29
6.3 Mecanismo de recomendación basado en popularidad.....30
Ejemplo de recomendaciones generadas.....31
6.4 Relación con Machine Learning.....31
6.5 Conclusión.....32
7. Opcional: Mejora en visualización.....33
8. Conclusión.....34

1. Introducción

En este proyecto se ha trabajado con diferentes conjuntos de datos de *reviews* de Amazon, cuyo objetivo principal ha sido diseñar, implementar y explotar unos conjuntos de datos combinando bases de datos relacionales y no relacionales. En concreto, se han utilizado cuatro datasets pertenecientes a distintas categorías de productos: *Toys and Games*, *Video Games*, *Digital Music* y *Musical Instruments*, cada uno compuesto por miles de reseñas realizadas por usuarios sobre distintos artículos.

Cada una de las *reviews* contiene información estructurada (identificadores de usuario y producto, puntuación, fechas, etc.) junto con información no estructurada (texto de la reseña y resumen), lo que convierte este problema en un caso ideal para combinar tecnologías como MySQL y MongoDB. Esta dualidad ha permitido diseñar un sistema eficiente donde los datos estructurados se gestionan mediante un modelo relacional, mientras que los datos textuales se almacenan en una base de datos orientada a documentos.

A lo largo del proyecto se han abordado diferentes etapas que cubren todo el ciclo de vida de los datos. En primer lugar, se ha realizado el diseño del modelo de datos y la carga eficiente de grandes volúmenes de información sin utilizar técnicas que requieran mantener el dataset completo en memoria. Posteriormente, se ha desarrollado una aplicación en Python para la consulta y visualización de los datos mediante distintas representaciones gráficas.

Además, se ha incorporado el uso de bases de datos en grafo mediante Neo4J, permitiendo analizar relaciones entre usuarios y productos, así como calcular similitudes entre usuarios en base a sus patrones de consumo. Esta parte ha sido especialmente relevante para entender la estructura relacional implícita en los datos.

En las fases finales del proyecto, se ha ampliado la base de datos con nuevos conjuntos de datos, asegurando la integridad y consistencia del sistema, y se han explorado técnicas de recomendación.

En conjunto, este proyecto permite integrar conocimientos de bases de datos relacionales, no relacionales, procesamiento de datos y sistemas de recomendación, ofreciendo una visión completa de cómo gestionar y explotar grandes volúmenes de información en un entorno realista.

Durante la realización del proyecto hemos trabajado siempre de forma presencial y conjunta. Aunque en algunos momentos nos hemos repartido pequeñas tareas para avanzar más rápido, hemos seguido trabajando de forma coordinada, revisando, entendiendo y ayudando en el trabajo de la otra en todo momento. Esto nos ha permitido tener una visión completa del proyecto y comprender todas sus partes, desde el diseño de las bases de datos hasta la implementación y el análisis final.

2. Diseño y carga de datos

En esta primera parte del proyecto se ha realizado el diseño de la base de datos y la carga de los datos a partir de los ficheros JSON proporcionados.

Tal y como se indica en el enunciado, se ha combinado el uso de MySQL y MongoDB, almacenando una parte de la información en cada una de ellas. Para ello, se ha analizado la estructura de los datos y se ha decidido cómo repartirlos entre ambas bases de datos.

Por un lado, en MySQL se han almacenado los datos estructurados, es decir, aquellos que tienen un formato fijo y se pueden organizar en tablas. En concreto, se han guardado los usuarios, los productos y las reviews, incluyendo campos como el identificador del usuario (*reviewerID*), el identificador del producto (*asin*), la puntuación (overall), los votos útiles (*helpful*), la fecha y el tiempo en formato Unix.

Por otro lado, en MongoDB se han almacenado los campos de texto libre, que son más variables y no siguen una estructura fija. En este caso, se han guardado el texto completo de la review (*reviewText*) y el resumen (*summary*), junto con los identificadores necesarios para relacionarlos con los datos almacenados en MySQL.

De esta forma, se cumple el requisito del enunciado de utilizar tanto bases de datos relacionales como no relacionales, aprovechando las ventajas de cada una según el tipo de datos.

2.1 Diseño del modelo relacional (MySQL)

En MySQL se ha diseñado un modelo relacional que permite organizar la información estructurada de los datos.

En concreto, se han creado tres tablas principales:

- *usuarios*: almacena la información de los usuarios que han realizado reviews.
- *productos*: almacena los artículos sobre los que se realizan las valoraciones.
- *reviews*: almacena cada una de las valoraciones realizadas por los usuarios.

La tabla *usuarios* utiliza como clave primaria el campo *reviewer_id*, mientras que la tabla *productos* utiliza el campo *asin*, que actúa como identificador único del artículo. En la tabla *reviews* se ha definido un identificador propio (*review_id*) como clave primaria autoincremental.

Además, la tabla *reviews* incluye dos claves foráneas que permiten relacionar la información:

- *reviewer_id*, que referencia a la tabla usuarios
- *asin*, que referencia a la tabla productos

De esta forma, un usuario puede realizar varias reviews y un producto puede tener múltiples valoraciones, sin necesidad de duplicar información.

También se han creado índices sobre los campos *reviewer_id, asin* y *review_time*, ya que son los más utilizados en las consultas, mejorando así el rendimiento.

Por último, el campo *reviewTime*, que en los ficheros JSON aparece como texto, se ha transformado antes de insertarlo para almacenarlo como tipo DATE en MySQL, cumpliendo así el requisito del enunciado.

A continuación, se muestra el esquema relacional diseñado, indicando las claves primarias y foráneas:

USUARIOS(reviewer_id, reviewer_name)
PRODUCTOS(asin, categoria)
REVIEWS(review_id, reviewer_id, asin, overall, helpful_yes, helpful_total, unix_review_time, review_time)
  reviewer_id FK REFERS USUARIOS (reviewer_id)
  asin FK REFERS PRODUCTOS (asin)

2.2 Diseño del modelo no relacional (MongoDB)

En MongoDB se ha diseñado un modelo no relacional orientado a documentos, en el que se almacena la parte menos estructurada de los datos.

En concreto, se ha utilizado una colección principal:

- *reviews_texto*: almacena la información textual de las reviews.

Cada documento de esta colección incluye los siguientes campos:

- *reviewerID*: identificador del usuario
- *asin*: identificador del producto
- *categoria*: categoría del producto
- *reviewText*: texto completo de la review
- *summary*: resumen de la review

A diferencia de MySQL, en MongoDB no es necesario definir un esquema fijo, lo que permite almacenar información textual de forma flexible, incluso cuando su longitud o contenido varía entre registros.

En este caso, se ha decidido almacenar en MongoDB los campos *reviewText* y *summary*, ya que son datos no estructurados que no encajan bien en un modelo relacional. Además, se incluyen también los identificadores (*reviewerID* y *asin*) y la categoría para poder relacionar fácilmente esta información con los datos almacenados en MySQL. Por último, un campo *_id* es generado automáticamente por MongoDB y actúa como identificador único de cada documento dentro de la colección.

A continuación, se muestra la estructura de los documentos almacenados en MongoDB:

REVIEWS_TEXTO {_id, reviewerID, asin, categoria, reviewText, summary}

2.3 Proceso de carga de datos

Una vez diseñado el modelo de datos, se ha implementado un programa en Python llamado $load_data.py$ para realizar la carga de los datos, cumpliendo todas las condiciones indicadas en el enunciado.

En primer lugar, el programa crea automáticamente la base de datos en MySQL junto con las tablas necesarias. Además, se conecta a MongoDB y prepara la colección donde se almacenarán los datos.

A continuación, se leen los ficheros de reviews. Tal y como exige el enunciado, los ficheros se procesan línea a línea, sin cargarlos completos en memoria y sin utilizar librerías como pandas. De esta forma, el programa puede trabajar correctamente incluso con ficheros muy grandes.

Para cada línea del fichero, el programa lee la review en formato JSON y extrae la información necesaria. Algunos campos se transforman antes de guardarlos, como la fecha, que se convierte al formato DATE para poder almacenarla correctamente en MySQL. Después, los datos se separan según dónde deben almacenarse: la parte estructurada se envía a MySQL, mientras que el texto de la review y el resumen se guardan en MongoDB.

Para mejorar el rendimiento, las inserciones no se hacen una a una, sino que se agrupan varias reviews en memoria y se insertan todas juntas cuando se alcanza un cierto tamaño. De esta forma, el proceso es mucho más rápido.

En el caso de MySQL, se tiene en cuenta que los mismos usuarios y productos pueden aparecer en distintos datasets, por lo que se evita insertar duplicados en estas tablas. Sin embargo, las reviews sí se almacenan siempre como nuevas, ya que cada valoración es un registro independiente. En MongoDB, los documentos también se insertan en grupos, siguiendo el mismo criterio de eficiencia.

Antes de empezar la carga, se eliminan los datos existentes en ambas bases de datos, para asegurar que todo se carga desde cero de forma consistente.

Por último, el programa recorre automáticamente todos los datasets definidos en el archivo de configuración, de manera que ejecutando el script se carga toda la información completa.

2.4 Archivo de configuración

Para facilitar la reutilización y adaptabilidad del sistema, se ha creado un fichero de configuración independiente llamado $configuracion.py$.

En este archivo se define toda la información necesaria para ejecutar el programa, como:

- credenciales de conexión a MySQL y MongoDB
- nombres de las bases de datos y colecciones
- rutas de los ficheros de datos
- tamaño de los lotes de inserción

De esta forma, es posible modificar la configuración del sistema sin necesidad de cambiar el código principal, cumpliendo así con los requisitos del enunciado.

3. Aplicación Python para el acceso y visualización de los datos

En esta segunda parte del proyecto se ha desarrollado una aplicación en Python que permite acceder a los datos almacenados en MySQL y MongoDB y representarlos mediante distintas visualizaciones gráficas.

Tal y como se indica en el enunciado, el objetivo es proporcionar al usuario una forma sencilla de explorar los datos mediante un menú interactivo, que permita seleccionar diferentes tipos de análisis y visualizar los resultados en forma de gráficos.

3.1 Estructura general de la aplicación

Para implementar esta funcionalidad, se ha desarrollado el fichero $menu_visualizacion.py$, que contiene un menú en terminal desde el cual el usuario puede seleccionar las distintas opciones disponibles.

El programa se ejecuta en un bucle continuo, de forma que el menú se muestra repetidamente hasta que el usuario selecciona la opción de salida. Esto permite realizar múltiples consultas y visualizaciones sin necesidad de reiniciar el programa.

La estructura del programa se basa en:

- Una función principal que gestiona el menú
- Una función independiente para cada tipo de gráfica
- Funciones auxiliares para la conexión a MySQL y MongoDB

Además, en aquellas opciones en las que el usuario debe elegir categoría, el programa muestra previamente las categorías disponibles obtenidas desde la base de datos, lo que ayuda a evitar errores al introducir los datos.

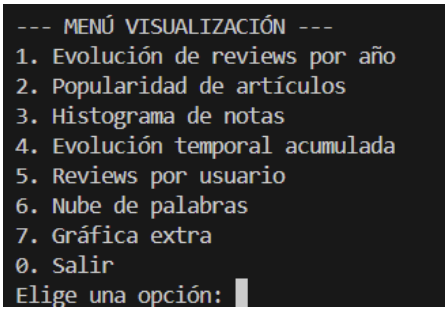


Figura 3.1: *Salida por terminal*

3.2 Tecnologías utilizadas

Para la implementación de esta parte se han utilizado las siguientes herramientas:

- MySQL: para obtener los datos estructurados necesarios para las gráficas
- MongoDB: para trabajar con el texto de las reviews en la nube de palabras
- Matplotlib: para la generación de todas las representaciones gráficas
- WordCloud: para la creación de la nube de palabras

El uso de estas herramientas permite combinar consultas eficientes sobre grandes volúmenes de datos con una representación visual clara de los resultados.

3.3 Evolución de reviews por año

El enunciado solicita obtener un histograma que muestre el número de reviews realizadas en cada año, permitiendo al usuario filtrar por tipo de producto o visualizar todos los datos conjuntamente.

Para implementar esta funcionalidad, el programa solicita al usuario una categoría o la opción “todo”. A continuación, se realiza una consulta a MySQL que agrupa las reviews por año utilizando la función YEAR sobre el campo de fecha.

En caso de seleccionar una categoría concreta, se realiza un JOIN con la tabla productos para filtrar únicamente los artículos pertenecientes a dicha categoría.

El resultado de la consulta se representa mediante un gráfico de barras, donde:

- el eje X representa los años
- el eje Y representa el número de reviews

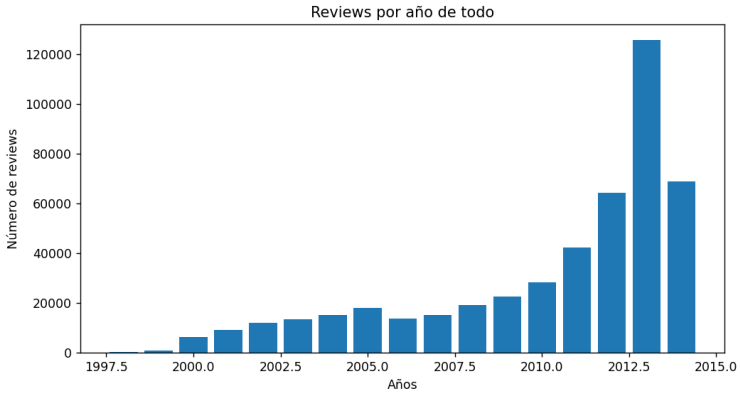


Figura 3.2: *Evolución de reviews por años*

Esta gráfica permite observar la evolución temporal de la actividad de los usuarios, identificando periodos de mayor o menor volumen de reviews.

3.4 Evolución de la popularidad de los artículos

En este apartado se pide representar la popularidad de los artículos, entendida como el número de reviews que tiene cada producto.

Para ello, el programa realiza una consulta que agrupa las reviews por artículo (asin) y calcula el número total de valoraciones para cada uno. Los resultados se ordenan de mayor a menor popularidad.

Al igual que en el caso anterior, se permite filtrar por categoría o visualizar todos los productos.

La representación se realiza mediante una curva donde:

- el eje X representa los artículos ordenados
- el eje Y representa el número de reviews

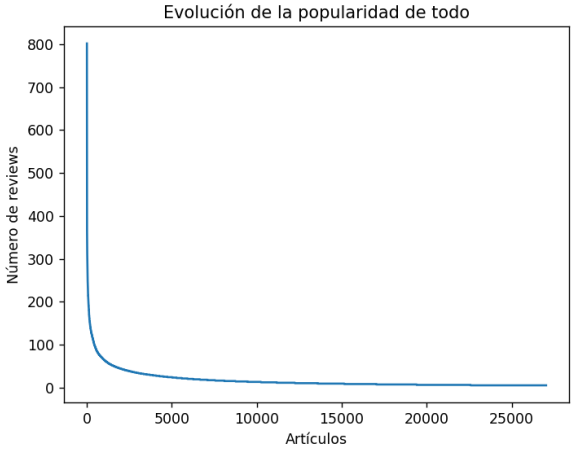


Figura 3.3: *Evolución de la popularidad de los productos*

Este gráfico permite observar la distribución de popularidad, identificando artículos muy valorados frente a otros con menor número de reviews.

3.5 Histograma por nota

El objetivo de esta funcionalidad es mostrar la distribución de las puntuaciones (overall) de las reviews.

El programa permite dos opciones: visualizar el histograma global o visualizarlo para un artículo concreto (introduciendo su asin).

En caso de introducir un asin, el programa comprueba previamente si existe en la base de datos, evitando errores.

La consulta agrupa las reviews por valor de nota y calcula cuántas veces aparece cada puntuación.

El resultado se representa mediante un histograma donde:

- el eje X representa las notas (1 a 5)
- el eje Y representa el número de reviews

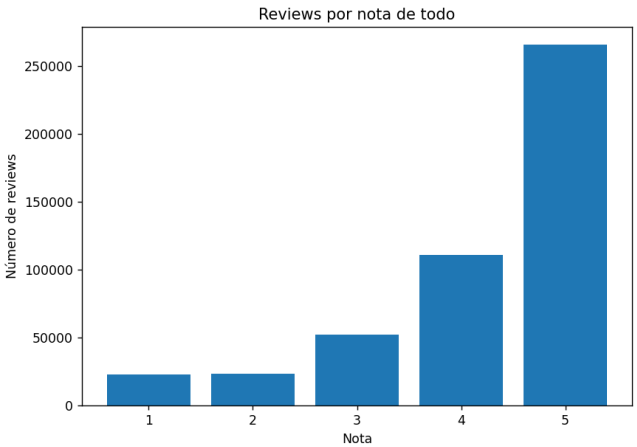


Figura 3.4: *Histograma por nota*

Esta gráfica permite analizar el comportamiento de los usuarios, observando si predominan valoraciones altas, bajas o distribuciones equilibradas.

3.6 Evolución de las reviews a lo largo del tiempo

En este apartado se pide mostrar la evolución acumulada del número de reviews a lo largo del tiempo.

Para ello, se obtiene el campo unix_review_time de las reviews y se ordena cronológicamente. A partir de esta secuencia, se construye una variable acumulada que representa el número total de reviews hasta cada instante.

El resultado se representa mediante una gráfica donde:

- el eje X representa el tiempo
- el eje Y representa el número acumulado de reviews

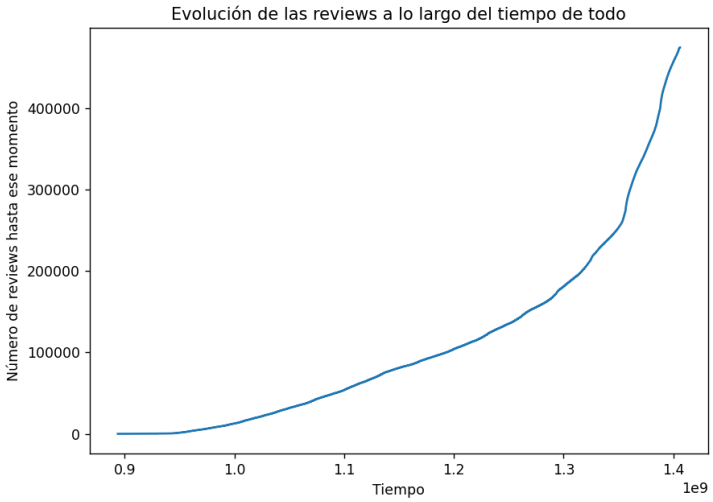


Figura 3.5: *Evolución de las reviews a lo largo del tiempo*

Esta gráfica permite visualizar el crecimiento del número de reviews, mostrando tendencias y posibles cambios en la actividad a lo largo del tiempo.

3.7 Histograma de reviews por usuario

Esta funcionalidad permite analizar el comportamiento de los usuarios en función del número de reviews que han realizado.

Para ello, se agrupan las reviews por usuario y se cuenta cuántas ha realizado cada uno. Posteriormente, se construye un histograma que muestra:

- en el eje X: número de reviews
- en el eje Y: número de usuarios con esa cantidad de reviews

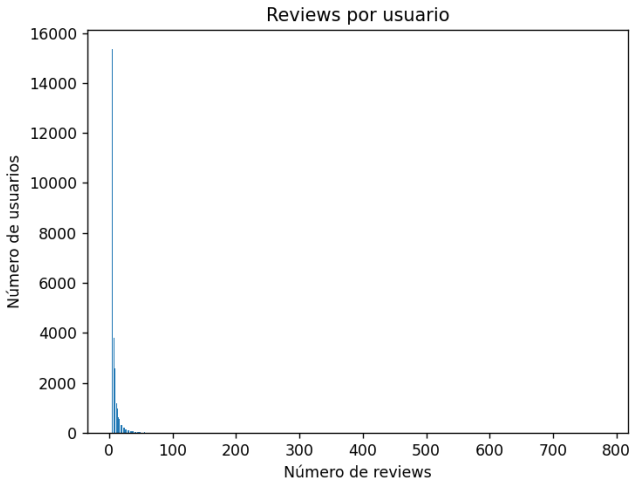


Figura 3.6: *Número de reviews por usuario*

Esta representación permite identificar patrones como la existencia de usuarios muy activos frente a la mayoría de usuarios con pocas contribuciones.

3.8 Nube de palabras

El enunciado solicita generar una nube de palabras a partir del campo summary, filtrando por categoría.

Para implementar esta funcionalidad, se consulta MongoDB para obtener los summaries de las reviews correspondientes a la categoría seleccionada.

Posteriormente:

- se separan las palabras
- se filtran aquellas con longitud menor o igual a 3
- se construye un texto completo con las palabras restantes

Este texto se utiliza como entrada para generar la nube de palabras mediante la librería WordCloud.


Nube de palabras del producto Toys_and_Games
Great
Awesome
Cute
Best
love
Good
Perfect
Nice
kid
play
fun
game
toy
durable
better
doll
Excellent
Lot
Adorable
Wonderful
Sturdy
puzzle
toddler
Lego
daughter
kids
Great gift
well made
Beautiful
Fantastic
Amazing
Five Star
Super
Pretty
Entertaining
Work
much
little
time
product
cheap
Simple
Classic
Another
Great addition
Great toy
Great quality
Great price
Great product
Great little
Great game
Great Great
Great Great Great
Great Great Great Great
Great Great Great Great Great
Great Great Great Great Great Great
Great Great Great Great Great Great Great
Great Great Great Great Great Great Great Great
Great Great Great Great Great Great Great Great Great
Great Great Great Great Great Great Great Great Great Great
Great Great Great Great Great Great Great Great Great Great Great
Great Great Great Great Great Great Great Great Great Great Great Great
Great Great Great Great Great Great Great Great Great Great Great Great Great
Great Great Great Great Great Great Great Great Great Great Great Great Great Great
Great Great Great Great Great Great Great Great Great Great Great Great Great Great Great
Great Great Great Great Great Great Great Great Great Great Great Great Great Great Great Great
Great Great Great Great Great Great Great Great Great Great Great Great Great Great Great Great Great
Great Great Great Great Great Great Great Great Great Great Great Great Great Great Great Great Great Great
Great Great Great Great Great Great Great Great Great Great Great Great Great Great Great Great Great Great Great
Great Great Great Great Great Great Great Great Great Great Great Great Great Great Great Great Great Great Great Great
Great Great
Great Great
Great Great
Great Great
Great Great
Great Great
Great Great
Great Great
Great Great
Great Great
Great Great
Great Great
Great Great
Great Great
Great

3.9 Gráfica adicional: media de puntuaciones por artículo

Como parte adicional, se ha implementado una gráfica que muestra la distribución de la media de puntuaciones por artículo.

Para ello, se calcula la media de la nota (overall) para cada producto y se representa mediante un histograma.

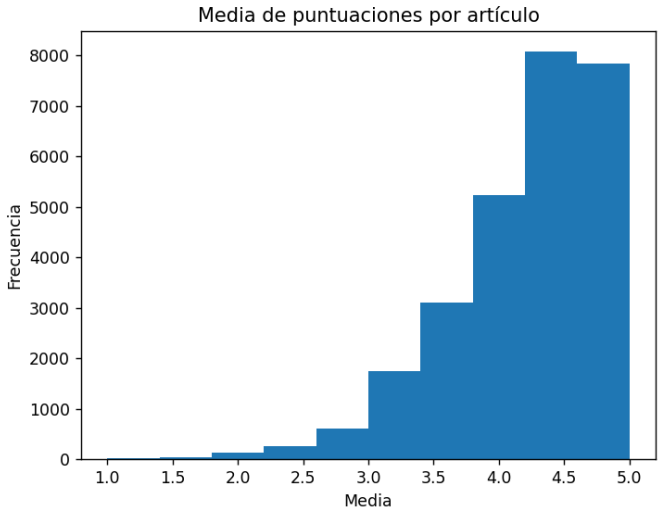


Figura 3.8: *Gráfica extra*

Esta gráfica permite analizar la calidad media de los productos, identificando si la mayoría presenta valoraciones altas o si existe una mayor dispersión.

3.10 Conclusión

En este apartado se ha desarrollado una herramienta completa de análisis y visualización que permite explorar los datos de forma interactiva.

La aplicación combina consultas eficientes a MySQL y MongoDB con representaciones gráficas claras, permitiendo obtener información relevante sobre el comportamiento de los usuarios, la popularidad de los productos y las características de las reviews.

En conjunto, esta parte del proyecto demuestra cómo es posible transformar grandes volúmenes de datos en información visual comprensible y útil para su análisis.

4. Aplicación Python junto con Neo4J

La tercera parte del proyecto se ha centrado en la incorporación de una base de datos orientada a grafos mediante Neo4J, con el objetivo de representar de forma visual e intuitiva relaciones que resultan más difíciles de analizar en un modelo exclusivamente relacional. Mientras que MySQL y MongoDB han permitido almacenar y consultar la información de las *reviews*, Neo4J aporta una capa adicional de análisis basada en conexiones entre entidades, especialmente útil para estudiar similitudes entre usuarios, relaciones entre usuarios y artículos, o vínculos entre usuarios y categorías.

Para esta parte se ha desarrollado el fichero $neo4JProyecto.py$, que contiene un menú con distintas opciones, cada una de ellas orientada a cargar en Neo4J un subconjunto concreto de información y representarlo como un grafo. En todos los casos, la lógica principal se implementa en Python, extrayendo previamente la información necesaria desde MySQL, transformándola al formato adecuado y cargándola después en Neo4J. De esta forma, Neo4J no se utiliza como base de datos principal del proyecto, sino como herramienta de análisis relacional y visualización de grafos, en línea con lo solicitado en el enunciado.

4.1 Obtener similitudes entre usuarios y mostrar los enlaces en Neo4J

En este primer subapartado se ha implementado una funcionalidad orientada a identificar relaciones de similitud entre usuarios a partir de sus valoraciones. Para ello, el programa selecciona en primer lugar los 30 usuarios con mayor número de *reviews*, valor que se ha dejado fácilmente modificable en el código para poder adaptarlo con cambios mínimos.

Una vez obtenidos estos usuarios, se recuperan desde MySQL todas las valoraciones realizadas por cada uno de ellos. Con esta información se construye en Python una estructura de datos en la que, para cada usuario, se almacena su identificador, su nombre y un diccionario de artículos puntuados junto con la nota otorgada. Esta representación intermedia permite trabajar de forma cómoda con los artículos valorados por cada usuario y calcular intersecciones entre pares de usuarios.

A continuación, se calcula la similitud entre cada par de usuarios mediante la correlación de Pearson, siguiendo la fórmula indicada en el enunciado. Para cada pareja, solo se almacena la similitud si ambos usuarios han puntuado al menos un artículo en común. Además de la propia similitud, se guarda también el número de artículos compartidos, ya que este dato resulta útil para interpretar la fortaleza real de la relación entre usuarios.

Antes de cargar la información en Neo4J, se limpia el grafo previamente generado para evitar interferencias con ejecuciones anteriores. Después, se crea un nodo por cada usuario, usando como propiedades su identificador original (reviewer_id) y su nombre (reviewer_name). Las similitudes calculadas se representan mediante relaciones SIMILITUD entre pares de usuarios, incluyendo como propiedades el valor de Pearson y el número de artículos comunes.

Por último, una vez cargado el grafo, se ejecuta una consulta en Neo4J para identificar qué usuario tiene más vecinos, es decir, qué nodo presenta un mayor número de relaciones de similitud con otros usuarios dentro del subconjunto analizado. Este resultado también se muestra por pantalla desde Python, cumpliendo así el requisito indicado en el enunciado.

Comprobaciones del código con consultas Neo4J

En este apartado veremos varias consultas ejecutadas en Neo4J con el objetivo de demostrar que el código de Python funciona correctamente.

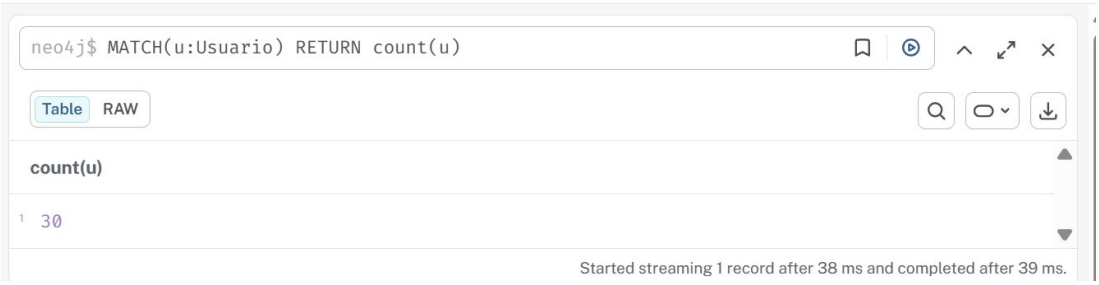


***Figura 4.1**: Número de usuarios cargados desde el programa de Python*

La primera comprobación realizada en Neo4J consiste en verificar que el número de nodos de tipo *Usuario* coincide con los 30 usuarios con mayor número de reviews seleccionados previamente desde MySQL. El resultado obtenido confirma que la carga de nodos se ha realizado correctamente y que no se han producido pérdidas de información durante el proceso de integración entre ambas bases de datos.



***Figura 4.2:** Número de relaciones SIMILITUD creadas*

A continuación, se comprueba el número total de relaciones *SIMILITUD* generadas. Este valor permite verificar que el cálculo de similitudes se ha realizado correctamente y que solo se han creado relaciones entre usuarios que comparten al menos un artículo puntuado. Además, el número de relaciones refleja la densidad del grafo generado, indicando un alto grado de interconexión entre los usuarios seleccionados.

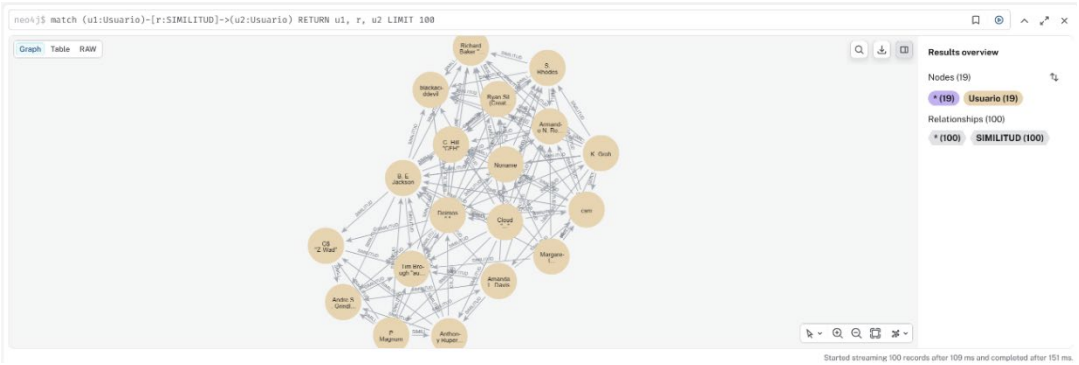


***Figura 4.3:** Grafo de similitudes*

La visualización del grafo permite observar cómo se estructuran las relaciones de similitud entre los 30 usuarios seleccionados. Cada nodo representa un usuario y cada enlace una relación *SIMILITUD*, cuya existencia indica que ambos usuarios han puntuado al menos un artículo en común.

La elevada densidad del grafo sugiere que existe un alto solapamiento en los patrones de consumo, lo que es coherente con la selección de usuarios más activos. Asimismo, pueden identificarse nodos con mayor número de conexiones, que actúan como puntos centrales dentro de la red, reflejando usuarios con comportamientos de valoración más representativos o compartidos.

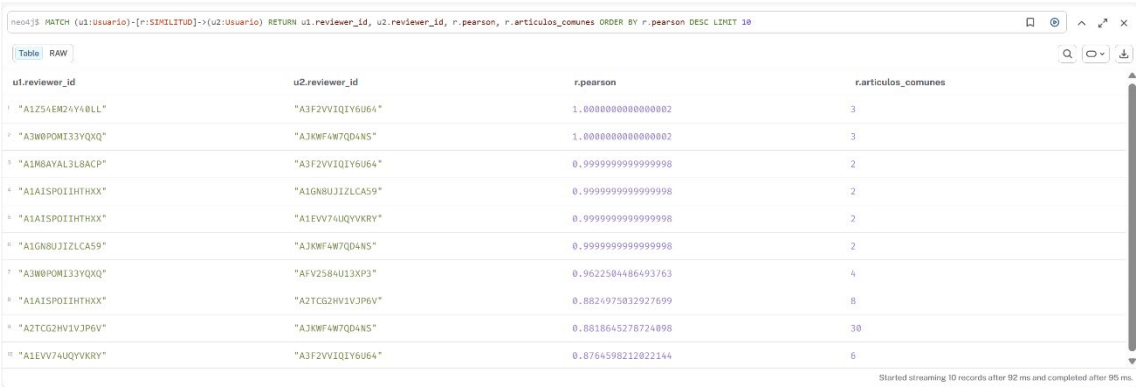


***Figura 4.4**: Valores Pearson y número de artículos comunes entre usuarios*

Además de la representación visual, se ha consultado una tabla con los valores numéricos asociados a cada relación. En concreto, se muestran tanto el coeficiente de correlación de Pearson como el número de artículos puntuados en común entre cada pareja de usuarios. Estos valores permiten interpretar con mayor precisión la intensidad de la similitud, ya que no solo indican si dos usuarios están conectados, sino también el grado de afinidad en sus valoraciones. Se observa que existen valores de correlación elevados, lo que confirma la existencia de patrones de comportamiento similares entre determinados usuarios.



***Figura 4.5:** Usuario con más vecinos*

Finalmente, se ejecuta una consulta para identificar el usuario con mayor número de vecinos dentro del grafo, es decir, aquel que presenta más relaciones de similitud con otros usuarios. Este resultado permite identificar usuarios altamente conectados, que pueden interpretarse como perfiles representativos dentro del conjunto analizado. Además, el resultado coincide con el mostrado por pantalla desde Python, lo que sirve como validación adicional del correcto funcionamiento de la integración y del cálculo de similitudes.

En conjunto, este apartado permite comprobar que Neo4J resulta especialmente útil para representar relaciones de similitud entre usuarios de forma visual y estructurada. Frente a una consulta relacional tradicional, el modelo de grafo facilita la identificación de usuarios muy conectados, así como el análisis de comunidades o patrones de comportamiento similares dentro del conjunto de datos.

4.2 Obtener enlaces entre usuarios y artículos

En este subapartado se ha desarrollado una funcionalidad orientada a representar en Neo4J la relación directa entre usuarios y artículos a partir de sus valoraciones. A diferencia del apartado

anterior, donde se modelaban relaciones entre usuarios, en este caso el objetivo es construir un grafo en el que intervienen dos tipos de nodos: usuarios y artículos.

Para ello, el programa solicita por pantalla una categoría de productos y un número de artículos a analizar. En nuestro caso, se ha trabajado con la categoría Toys_and_Games, (aunque se podría seleccionar cualquiera de las 4) seleccionando 50 artículos aleatorios. Esta selección aleatoria permite trabajar con subconjuntos distintos del dataset en cada ejecución, facilitando el análisis exploratorio de los datos.

Una vez definidos estos parámetros, se realiza una consulta a MySQL para obtener los identificadores (*ASIN*) de los artículos pertenecientes a la categoría seleccionada. Posteriormente, se recuperan todas las *reviews* asociadas a esos artículos, incluyendo información tanto del usuario como del producto y de la propia valoración (nota y momento temporal).

Antes de cargar los datos en Neo4J, se elimina el contenido previo del grafo para evitar mezclar información de distintas ejecuciones. A continuación, se construye el grafo siguiendo el siguiente esquema:

- Se crea un nodo Usuario por cada usuario que ha valorado alguno de los artículos seleccionados.
- Se crea un nodo Articulo por cada uno de los productos.
- Se establece una relación PUNTUO entre usuario y artículo, que representa la acción de haber realizado una valoración.

Además, esta relación almacena propiedades adicionales relevantes:

- overall: la puntuación otorgada por el usuario.
- unix_review_time: el instante temporal de la valoración.

De este modo, el grafo no solo refleja la existencia de una interacción entre usuario y artículo, sino también información cuantitativa y temporal asociada a dicha interacción.

Finalmente, una vez completada la carga, el usuario puede explorar visualmente el grafo en Neo4J, analizando patrones de comportamiento, usuarios más activos o artículos más valorados.

Comprobaciones del código con consultas Neo4J

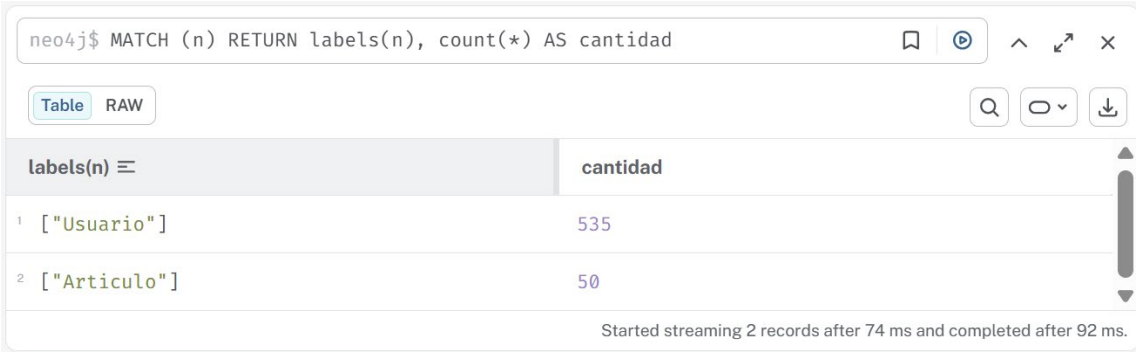


Figura 4.6: *Comprobación del número de nodos de usuarios y artículos*

En primer lugar, se ha comprobado la correcta creación de los nodos en Neo4J. El resultado muestra la existencia de dos tipos de nodos (*Usuario* y *Articulo*), lo que confirma que la

información originalmente almacenada en MySQL se ha transformado adecuadamente a un modelo de grafo.

Además, esta estructura refleja un grafo, en el que los usuarios se conectan exclusivamente con artículos, lo que resulta coherente con la naturaleza de los datos de valoraciones.

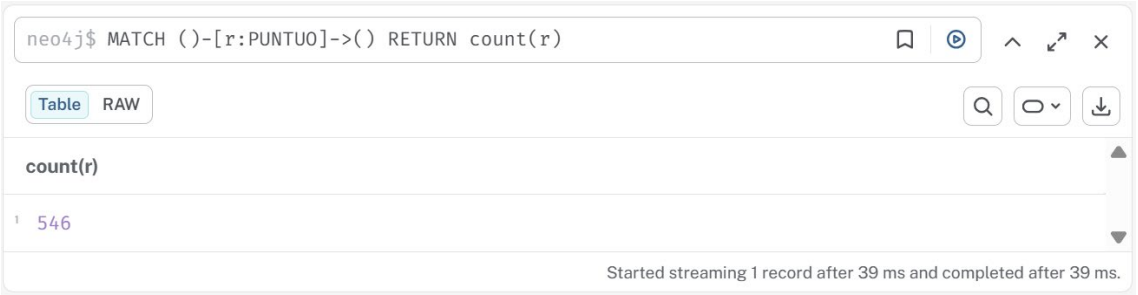


Figura 4.7: *Número de relaciones*

En esta imagen se ha verificado el número total de relaciones del tipo *PUNTUO* generadas. Este valor refleja el número de interacciones usuario-artículo cargadas en el grafo, confirmando que las *reviews* han sido correctamente representadas como relaciones.
La magnitud del número de relaciones es coherente con el número de usuarios y artículos cargados, lo que indica que el proceso de inserción ha capturado adecuadamente las valoraciones disponibles para los artículos seleccionados.



Figura 4.8: *Visualización de relaciones entre usuarios y artículos*

La visualización del grafo permite observar la estructura de relaciones entre usuarios y artículos. Se aprecia claramente la naturaleza bipartita del grafo, donde los nodos de tipo *Usuario* se conectan únicamente con nodos de tipo *Articulo* mediante relaciones *PUNTUO*.

La distribución de conexiones permite identificar artículos con mayor número de valoraciones (nodos más conectados), así como usuarios más activos dentro del subconjunto analizado.

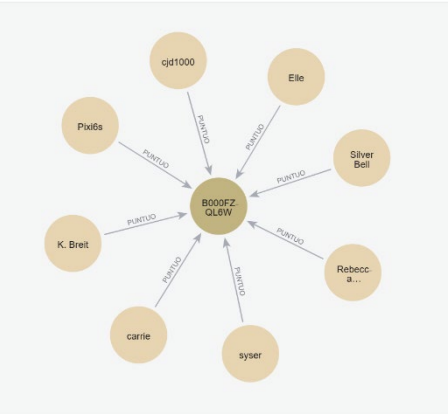
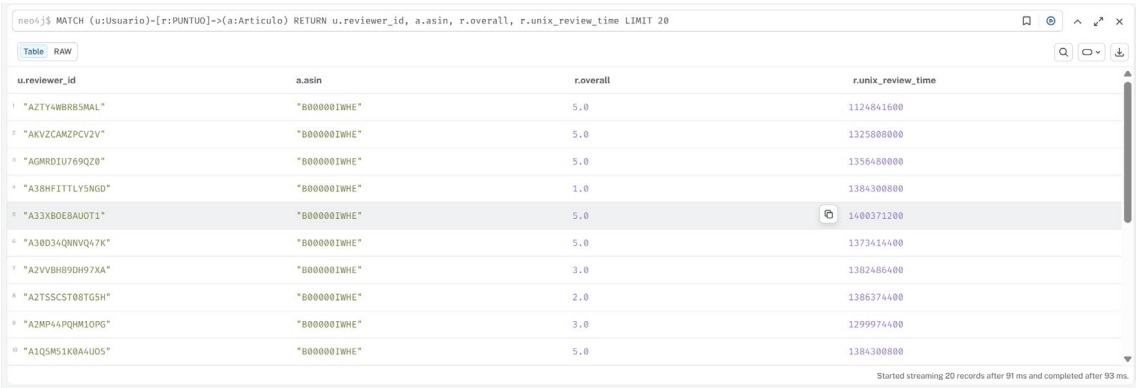


***Figura 4.9**: Zoom en uno de los grafos anteriores*

En la segunda imagen se presenta un zoom sobre una parte concreta del grafo, lo que permite analizar con mayor detalle las relaciones entre un artículo y los usuarios que lo han valorado. Este tipo de visualización facilita la identificación de patrones locales, como la concentración de valoraciones en determinados artículos, así como el comportamiento individual de los usuarios en relación con un producto específico.



```
neo4j$ MATCH (u:Usuario)-[r:PUNTUO]->(a:Articulo) RETURN u.reviewer_id, a.asin, r.overall, r.unix_review_time LIMIT 20
```

Table RAW

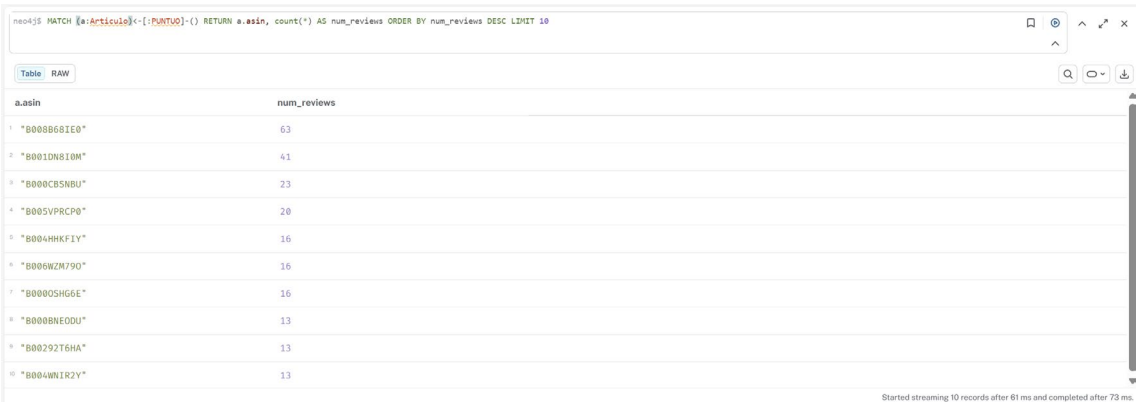
	u.reviewer_id	a.asin	r.overall	r.unix_review_time
1	"AZTY4WBRB5MAL"	"B00000IWHE"	5.0	1124841600
2	"AKVZCAMZPCV2V"	"B00000IWHE"	5.0	1325808000
3	"AGMRDIU769QZ0"	"B00000IWHE"	5.0	1356480000
4	"A38HFITTLY5NGD"	"B00000IWHE"	1.0	1384300800
5	"A33XBOE8AUOT1"	"B00000IWHE"	5.0	1400371200
6	"A30D34QNNVQ47K"	"B00000IWHE"	5.0	1373414400
7	"A2VVBH89DH97XA"	"B00000IWHE"	3.0	1382486400
8	"A2TSSCST08TGSH"	"B00000IWHE"	2.0	1386374400
9	"A2MP44PQHM1OPG"	"B00000IWHE"	3.0	1299974400
10	"A1Q5M51K0A4UO5"	"B00000IWHE"	5.0	1384300800

Started streaming 20 records after 91 ms and completed after 93 ms.

***Figura 4.10**: Propiedades de las relaciones*

Además de la conectividad, se ha comprobado que las relaciones *PUNTUO* almacenan correctamente las propiedades asociadas a cada valoración. En concreto, se incluyen tanto la puntuación otorgada (*overall*) como el instante temporal en el que se realizó la *review* (*unix_review_time*).

Esto permite enriquecer el análisis posterior, ya que no solo se dispone de información estructural, sino también de información semántica y temporal relevante.



```
neo4j$ MATCH (a:Articulo)<-[:PUNTUO]-() RETURN a.asin, count(*) AS num_reviews ORDER BY num_reviews DESC LIMIT 10
```

Table RAW

	a.asin	num_reviews
1	"B008B68IE0"	63
2	"B001DN8I0M"	41
3	"B000CB5NBU"	23
4	"B005VPRCP0"	20
5	"B004HHKFIY"	16
6	"B006WZM79O"	16
7	"B000OSHG6E"	16
8	"B000BNEODU"	13
9	"B00292T6HA"	13
10	"B004WNIR2Y"	13

Started streaming 10 records after 61 ms and completed after 73 ms.

***Figura 4.11:** Artículos más populares*

Finalmente, se ha realizado una consulta para identificar los artículos con mayor número de valoraciones dentro del subconjunto analizado. Este análisis permite detectar productos especialmente populares, es decir, aquellos que concentran un mayor número de interacciones por parte de los usuarios.

Además, sirve como validación adicional de la coherencia de los datos cargados, ya que los artículos con mayor conectividad en el grafo coinciden con los resultados obtenidos mediante consultas agregadas.

Por lo tanto, este apartado demuestra cómo Neo4J permite representar de forma natural las interacciones entre usuarios y artículos, integrando en una misma estructura tanto la relación como sus propiedades asociadas. Este enfoque facilita el análisis exploratorio de los datos y sienta bases para posibles sistemas de recomendación basados en grafos.

4.3 Obtener algunos usuarios que han visto más de un determinado tipo de artículo

En este subapartado se ha trabajado con un enfoque distinto a los anteriores, centrado no en artículos individuales ni en similitudes entre usuarios, sino en el comportamiento de consumo por categorías.

Para ello, se han seleccionado los primeros 400 usuarios ordenados alfabéticamente por nombre, tal y como indica el enunciado. Sobre este subconjunto, se ha analizado qué usuarios han interactuado con artículos pertenecientes a más de una categoría, es decir, usuarios con un patrón de consumo diverso.

A partir de este filtrado, se construye un grafo en Neo4J con dos tipos de nodos:

- Usuario
- TipoArticulo (representando categorías como *Toys_and_Games*, *Video_Games*, etc.)

Las relaciones entre ambos (CONSUME) indican que un usuario ha puntuado artículos de una determinada categoría. Además, cada relación incorpora una propiedad clave:

- num_articulos: número de artículos de esa categoría que el usuario ha puntuado

Este diseño permite no solo identificar si un usuario ha interactuado con una categoría, sino también cuánto lo ha hecho, aportando una visión cuantitativa del comportamiento.

De esta forma, el grafo resultante permite analizar patrones de consumo más complejos, como usuarios multidisciplinares, especialización en ciertas categorías o afinidades entre tipos de productos.

Comprobaciones del código con consultas Neo4J

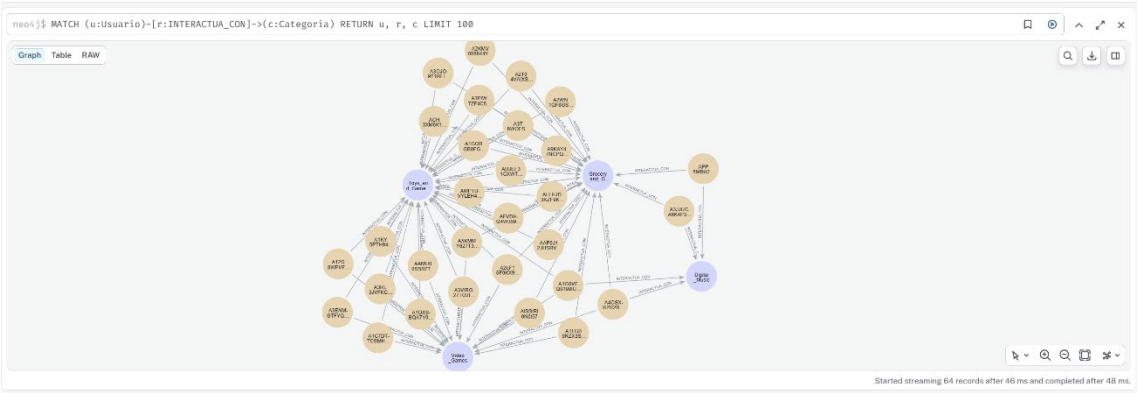


Figura 4.12: *Estructura global del grafo*

La visualización del grafo permite observar la relación entre usuarios y categorías de artículos. A diferencia de apartados anteriores, en este caso el análisis se realiza a un nivel más agregado, donde los nodos representan tipos de producto en lugar de artículos individuales. Se aprecia cómo determinados usuarios se conectan a varias categorías, lo que indica un comportamiento de consumo diverso. Este tipo de representación resulta especialmente útil para identificar perfiles de usuario con intereses amplios y patrones de consumo menos especializados.

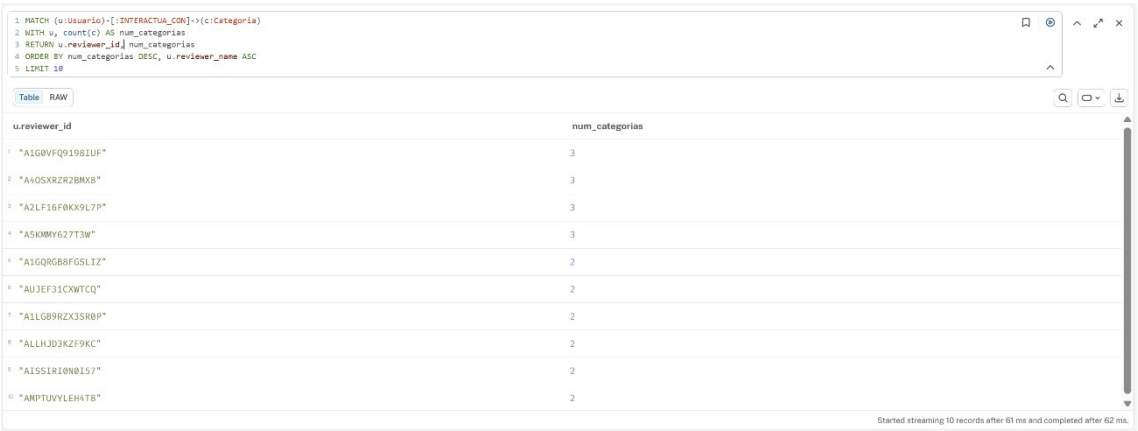


Figura 4.13: *Usuarios con mayor diversidad de categorías*

Esta consulta permite identificar los usuarios con mayor diversidad de consumo dentro del subconjunto analizado, midiendo el número de categorías distintas con las que interactúan. El resultado confirma que los usuarios cargados en Neo4J cumplen la condición del enunciado, es decir, no se trata de usuarios especializados en una única categoría, sino de perfiles con actividad distribuida entre distintos tipos de artículos.

Esto valida que el proceso de filtrado previo se ha realizado correctamente.

```
MATCH (u:Usuario)-[r:INTERACTUA_CON]->(c:Categoria)
RETURN u.reviewer_id,c.nombre, r.num_articulos
ORDER BY r.num_articulos DESC, u.reviewer_name ASC
LIMIT 10
```

Table RAW

	u.reviewer_id	c.nombre	r.num_articulos
1	"A1G0VFQ9198IUF"	"Video_Games"	118
2	"ALLHJD3KZF9KC"	"Toys_and_Games"	54
3	"A4OSXRZR2BMX8"	"Digital_Music"	35
4	"A12G8WFVFU2RGO"	"Video_Games"	30
5	"APP5MBH2BSX6I"	"Grocery_and_Gourmet_Foods"	28
6	"A3ENMGTFYQVTHB"	"Toys_and_Games"	25
7	"A2WN1QF8GSVHYV"	"Grocery_and_Gourmet_Foods"	23
8	"A3PIW7ZF4C5ZJL"	"Toys_and_Games"	21
9	"A2LF16F0KX9L7P"	"Video_Games"	20
10	"A2LF16F0KX9L7P"	"Grocery_and_Gourmet_Foods"	19

Started streaming 10 records after 56 ms and completed after 57 ms.

***Figura 4.14**: Intensidad de consumo por categoría*

Además de reflejar si un usuario ha interactuado con una categoría, el grafo almacena la intensidad de dicha interacción mediante la propiedad *num_articulos*.

Esta consulta permite identificar no solo la diversidad de consumo, sino también el grado de implicación de cada usuario en cada categoría. Se observan casos en los que un usuario concentra una parte significativa de su actividad en una categoría concreta, lo que permite distinguir entre usuarios generalistas y usuarios con preferencias dominantes. Este enfoque aporta una visión más rica que una simple relación binaria usuario-categoría.
Este apartado permite pasar de un análisis centrado en artículos concretos a un análisis del comportamiento global de los usuarios en función de las categorías.

El uso de Neo4J resulta especialmente útil en este contexto, ya que facilita la representación y exploración de relaciones de distinta granularidad, permitiendo identificar tanto la diversidad como la intensidad del consumo.

En conjunto, este enfoque proporciona una comprensión más completa de los patrones de comportamiento de los usuarios, alineándose con el objetivo del enunciado.

4.4 Artículos populares y artículos en común entre usuarios

En este último subapartado de la parte de Neo4J se ha construido un grafo mixto que combina, por un lado, la relación entre usuarios y artículos, y por otro, la relación entre usuarios en función de los artículos que tienen en común.

Para ello, el programa comienza seleccionando desde MySQL los 5 artículos más populares que tengan menos de 40 reviews. Esta restricción resulta interesante porque evita quedarse con artículos extremadamente populares, permitiendo analizar productos que, aun no teniendo un volumen masivo de interacciones, generan suficientes conexiones entre usuarios como para construir un grafo significativo.

Una vez obtenidos estos 5 artículos, se recuperan todas las *reviews* asociadas a ellos, incluyendo para cada una el usuario, el artículo, la categoría, la nota otorgada y el instante temporal de la valoración. Con esta información se genera en Neo4J un primer nivel del grafo formado por:

- nodos Usuario

- nodos Articulo
- relaciones PUNTUO entre ambos

Cada relación PUNTUO almacena como propiedades:

- overall: puntuación otorgada por el usuario
- unix_review_time: momento temporal de la review

Posteriormente, se construye un segundo nivel de relaciones entre usuarios. Para ello, se analiza qué artículos de esos cinco ha puntuado cada usuario y se calcula, para cada pareja, el número de artículos que tienen en común. Si el número de coincidencias es mayor que cero, se crea entre ellos una relación TIENE_EN_COMUN, cuya propiedad articulos_comunes indica cuántos de esos artículos han sido puntuados por ambos usuarios.

De esta manera, el grafo final permite representar simultáneamente:

- la interacción directa usuario-artículo
- y la afinidad entre usuarios basada en consumo compartido

Este enfoque resulta especialmente útil para detectar pequeños grupos de usuarios conectados entre sí por intereses comunes, lo que lo convierte en una representación muy adecuada para análisis exploratorio y para sentar bases de posibles sistemas de recomendación.

Comprobaciones del código con consultas Neo4J

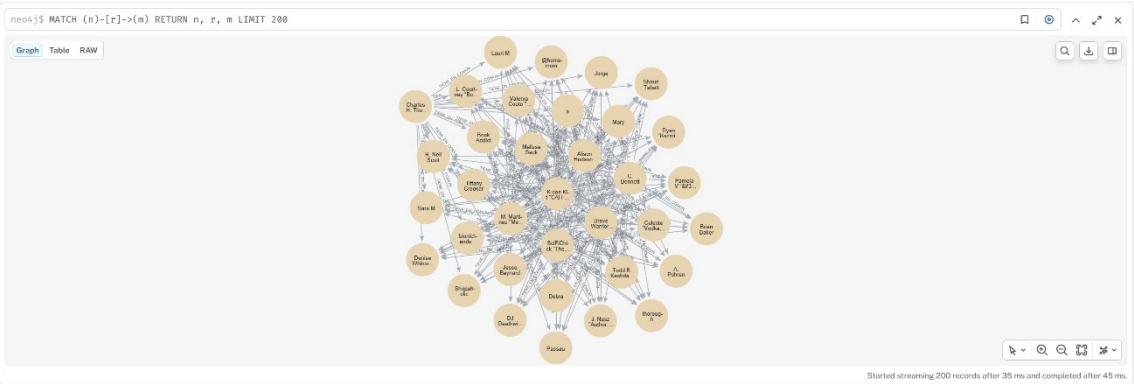


***Figura 4.15**: Visualización general del grafo*

La visualización general del grafo permite observar simultáneamente los 5 artículos seleccionados, los usuarios que los han puntuado y las relaciones de artículos en común entre usuarios.
Esta representación integra en una única estructura tanto las relaciones directas usuario-artículo (*PUNTUO*) como las relaciones indirectas entre usuarios (*TIENE_EN_COMUN*), derivadas de sus intereses compartidos.

De este modo, se obtiene una visión más completa del sistema, en la que se combinan distintos niveles de información dentro del mismo grafo.

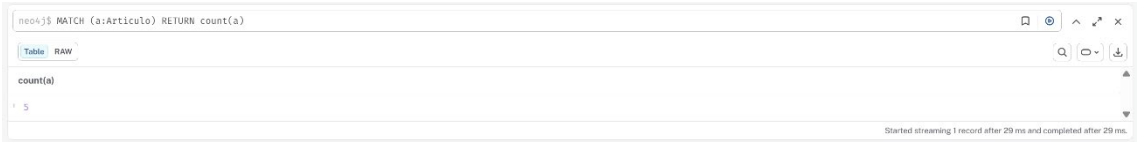


Figura 4.16: *Comprobación de carga de 5 artículos*

Se ha verificado en Neo4J que el número de nodos de tipo *Articulo* cargados en el grafo es exactamente 5, en coherencia con la selección realizada previamente en MySQL de los cinco artículos más populares con menos de 40 *reviews*.

Este resultado confirma que el filtrado y la transferencia de datos se han ejecutado correctamente.



Figura 4.17: *Relaciones entre usuarios y número de artículos en común*

La consulta sobre las relaciones *TIENE_EN_COMUN* permite identificar pares de usuarios que han puntuado varios de los artículos seleccionados. La propiedad *articulos_comunes* cuantifica el número de coincidencias entre usuarios, proporcionando una medida directa del grado de similitud en su comportamiento.

Este tipo de información resulta especialmente relevante, ya que permite detectar usuarios con intereses similares dentro del subconjunto analizado y facilita la identificación de posibles agrupaciones o comunidades de usuarios.

Este apartado combina de forma especialmente clara las ventajas del modelo de grafos, ya que permite representar en una misma estructura tanto las valoraciones directas de los usuarios sobre los artículos como las relaciones indirectas entre usuarios derivadas de intereses compartidos.
Como resultado, Neo4J facilita una interpretación más rica, visual e intuitiva de las conexiones presentes en los datos, superando las limitaciones de los modelos relacionales tradicionales. En conjunto, este enfoque demuestra el potencial de los grafos para analizar relaciones complejas y extraer conocimiento significativo a partir de los datos.

4.5 Conclusión

A lo largo de este apartado se ha puesto de manifiesto el valor de Neo4J como herramienta complementaria a los modelos de datos tradicionales, permitiendo representar de forma

natural relaciones complejas entre entidades que, en un entorno puramente relacional, resultarían más difíciles de analizar e interpretar.

Partiendo de los datos almacenados en MySQL, se ha construido una serie de grafos que abordan distintas perspectivas del problema. En primer lugar, se han modelado similitudes entre usuarios mediante la correlación de Pearson, lo que ha permitido identificar conexiones basadas en patrones de valoración similares. Posteriormente, se ha representado la interacción directa entre usuarios y artículos, incorporando además información cuantitativa y temporal en las relaciones. A continuación, se ha dado un paso más hacia la abstracción, analizando el comportamiento de los usuarios a nivel de categorías, lo que ha permitido identificar perfiles de consumo diversos. Finalmente, se ha combinado la información de usuarios y artículos con relaciones entre usuarios basadas en consumo compartido, obteniendo una representación especialmente rica del conjunto de datos.

En conjunto, estos grafos no solo permiten visualizar la estructura de las relaciones presentes en los datos, sino que también facilitan la identificación de patrones relevantes, como usuarios altamente conectados, artículos que actúan como puntos de unión entre usuarios o perfiles de consumo diferenciados. Este tipo de análisis resulta especialmente útil en contextos como los sistemas de recomendación, donde la comprensión de las relaciones entre usuarios y productos es clave.

Por tanto, este apartado demuestra que la incorporación de un modelo de grafos no sustituye a las bases de datos relacionales o documentales utilizadas previamente, sino que las complementa, aportando una nueva dimensión de análisis que resulta especialmente potente cuando se trabaja con datos altamente interconectados.

5. Nuevos datos

En esta parte del proyecto se ha abordado la incorporación de un nuevo conjunto de datos adicional a la base ya construida previamente. En concreto, se ha trabajado con el dataset Grocery_and_Gourmet_Foods_5.json, perteneciente a una categoría distinta de las cuatro utilizadas en la carga inicial.

A diferencia de la fase de carga principal, en este caso no se trataba de reconstruir toda la infraestructura de datos desde cero, sino de diseñar un mecanismo que permitiera insertar un nuevo dataset de forma incremental sobre una base ya existente. Este requisito resulta especialmente relevante, ya que en un escenario real los datos no suelen cargarse una única vez, sino que la base debe poder ampliarse con nuevas fuentes o categorías manteniendo la coherencia del sistema.

Para ello, se ha implementado el fichero $inserta_dataset.py$, cuya función principal consiste en procesar el nuevo dataset línea a línea e incorporar su información tanto a MySQL como a MongoDB. El diseño adoptado mantiene la misma filosofía seguida en el resto del proyecto:

- MySQL almacena la parte estructurada de las *reviews*
- MongoDB almacena la parte textual

Además, como la base ya contenía información previa, ha sido necesario introducir un mecanismo de control de duplicados que impida insertar varias veces la misma review en caso de reejecución del script. De este modo, la funcionalidad desarrollada no solo amplía la base de datos, sino que lo hace de forma robusta, incremental y consistente.

```
150000 reviews procesadas...
151000 reviews procesadas...
Inserción del nuevo dataset completada correctamente.
Reviews leídas: 151254
Reviews nuevas insertadas: 151254
Reviews duplicadas detectadas: 0
```

***Figura 5.1**: Salida por terminal de la ejecución de insert_dataset.py*

La salida final del programa confirma que la inserción incremental del nuevo dataset se ha realizado correctamente. Además, se muestran tres indicadores relevantes: el número total de reviews leídas, las reviews nuevas efectivamente insertadas y las reviews duplicadas detectadas. Esta última magnitud resulta especialmente importante, ya que valida el mecanismo diseñado para evitar inserciones repetidas en caso de reejecución del script.

categoria	num_productos
Toys_and_Games	11885
Video_Games	10672
Musical_Instruments	900
Digital_Music	3568
Grocery_and_Gourmet_Foods	8713

***Figura 5.2**: Consulta MySQL mostrando que existe una nueva categoría*

Esta consulta en MySQL permite comprobar que la nueva categoría Grocery_and_Gourmet_Foods ha sido incorporada correctamente a la tabla de productos. El resultado confirma que la base de datos no se ha reconstruido desde cero, sino que se ha ampliado de forma incremental, manteniendo al mismo tiempo las categorías previamente cargadas.

asin	categoria	reviewer_id	overall	unix_review_time
616719923X	Grocery_and_Gourmet_Foods	A1VEELTKS8NLZB	4	1370044800
616719923X	Grocery_and_Gourmet_Foods	A14R9XMZVJ6INB	3	1400457600
616719923X	Grocery_and_Gourmet_Foods	A27IQHDZFQFNGG	4	1381190400
616719923X	Grocery_and_Gourmet_Foods	A31QY5TASILE89	5	1369008000
616719923X	Grocery_and_Gourmet_Foods	A2LWK003FFMCI5	4	1369526400
616719923X	Grocery_and_Gourmet_Foods	A1NZJTY0BAA2SK	4	1378339200
616719923X	Grocery_and_Gourmet_Foods	AA95FYFIP38RM	3	1382054400
616719923X	Grocery_and_Gourmet_Foods	A3FIVHUOGMUMPK	5	1372982400
616719923X	Grocery_and_Gourmet_Foods	A27FSPAMTQF1J8	5	1371168000
616719923X	Grocery_and_Gourmet_Foods	A33NXNZ79H5K51	1	1348012800

Figura 5.3: *Consulta en MySQL mostrando ejemplos del nuevo dataset*

La consulta muestra ejemplos de reviews pertenecientes a la nueva categoría almacenadas en MySQL. Se observa que se han insertado correctamente tanto la relación entre usuario y producto como los atributos estructurados asociados a cada valoración, como la puntuación y el instante temporal.

En este apartado se ha conseguido ampliar la base de datos del proyecto mediante la incorporación de un nuevo dataset sin necesidad de reconstruir toda la infraestructura previa. La solución implementada permite insertar de forma incremental una nueva categoría de productos, manteniendo la separación entre datos estructurados y no estructurados establecida en el diseño original.

Además, se ha introducido un mecanismo de control de duplicados que garantiza la consistencia de la base en caso de ejecutar el proceso más de una vez, lo que aporta robustez a la solución. El uso de inserciones por lotes permite, además, mantener un rendimiento adecuado incluso trabajando con un volumen elevado de datos.

En conjunto, este apartado demuestra que la arquitectura diseñada no solo es capaz de cargar datos inicialmente, sino también de evolucionar y ampliarse con nuevas fuentes de información, reproduciendo un escenario mucho más cercano al funcionamiento real de un sistema de datos.

6. Más visualización y relación con Machine Learning

En esta última parte del proyecto se abordan distintos aspectos relacionados con la visualización de los datos y su posible aplicación en modelos de recomendación, dando respuesta a los tres objetivos planteados en el enunciado: la representación de algunas de las gráficas mediante una herramienta externa a Python, la descripción de un posible modelo de Machine Learning y la implementación de un sistema de recomendación básico.

6.1 Visualización con herramienta externa

Aunque en la segunda parte del proyecto se han generado todas las gráficas mediante Python, en esta sección se han seleccionado tres de ellas para reproducirlas utilizando Tableau, tal y como se indica en el enunciado. El objetivo ha sido comprobar cómo los mismos datos pueden representarse en una herramienta orientada al análisis visual, permitiendo una interpretación más clara y facilitando la exploración de la información.

Para ello, se ha seguido un proceso en dos fases. En primer lugar, se han preparado los datos en formato CSV mediante la ejecución del script $generar_archivos.py$, cuyo objetivo es extraer la información necesaria desde las bases de datos ya construidas en apartados anteriores y transformarla en ficheros fácilmente utilizables en herramientas de visualización.

A partir de este proceso, se han generado ficheros independientes para cada tipo de análisis. En concreto, se han utilizado los siguientes ficheros:

- $reviews_por_anio.csv$: evolución temporal de las reviews
- $popularidad.csv$: número de reviews por artículo
- $nube_palabras.csv$: frecuencia de palabras en los textos

Estos ficheros se han importado posteriormente en Tableau como fuentes de datos, creando una hoja de trabajo para cada visualización. En cada caso, la gráfica se ha construido arrastrando las variables correspondientes a filas, columnas o propiedades visuales (como tamaño o color), lo que permite definir el gráfico de forma intuitiva sin necesidad de programación.

Evolución de reviews por año

Para esta gráfica se ha utilizado el fichero $reviews_por_anio.csv$.

En Tableau se ha construido una gráfica de barras en la que el eje horizontal representa los años y el eje vertical el número de reviews. Para ello, se ha asignado la variable de año a columnas y el número de reviews a filas, seleccionando posteriormente el tipo de gráfico de barras.

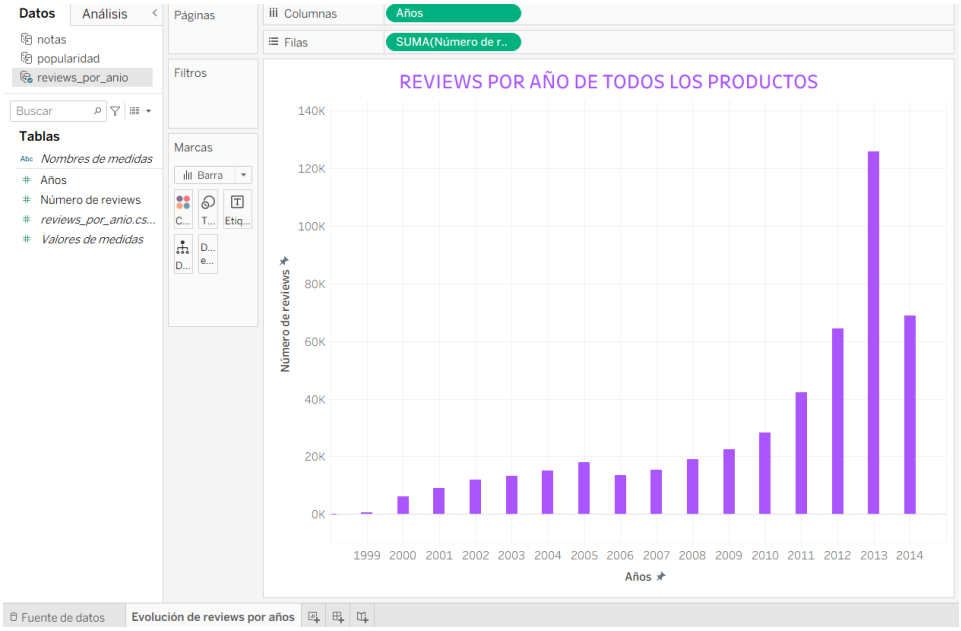


Figura 6.1: *Evolución de reviews por año*

El resultado muestra una tendencia claramente creciente, con un aumento significativo en los últimos años del dataset. Esto indica una mayor actividad de los usuarios a lo largo del tiempo.

Evolución de la popularidad de los artículos

Para esta visualización se ha utilizado el fichero $popularidad.csv$.

En este caso, se ha representado una gráfica de líneas en la que el eje horizontal corresponde a los artículos ordenados por popularidad (mediante la variable de posición), y el eje vertical indica el número de reviews. En Tableau, se ha asignado la variable de posición a columnas y el número de reviews a filas, seleccionando una representación en forma de línea para visualizar mejor la tendencia.

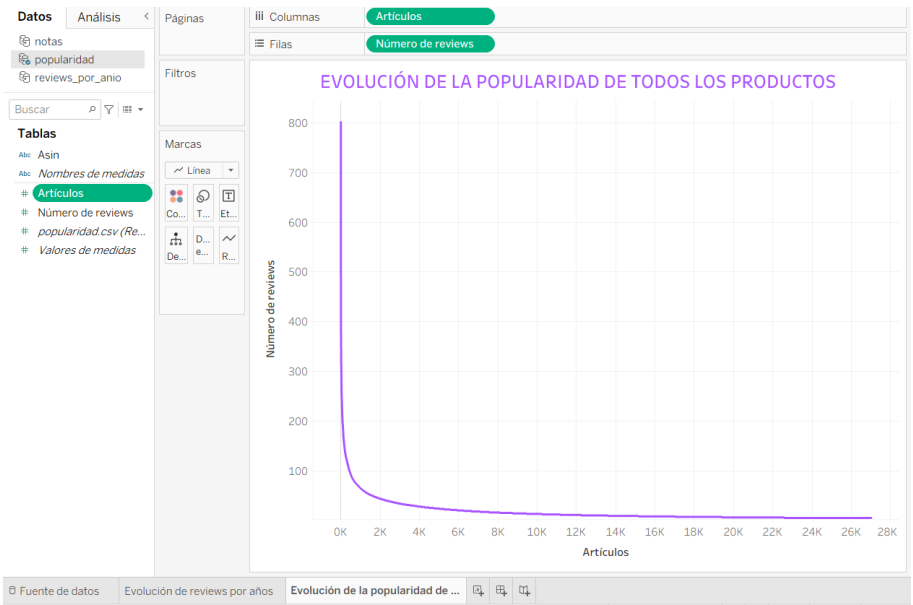


Figura 6.2: *Evolución de la popularidad de los artículos*

La forma de la curva presenta una caída muy pronunciada al inicio y una disminución progresiva posteriormente. Esto refleja que unos pocos artículos concentran un gran número de reviews, mientras que la mayoría tiene valores mucho menores, lo que corresponde a una distribución tipo “long tail”.

Nube de palabras

Para esta gráfica se ha utilizado el fichero $nube_palabras.csv$, generado a partir de los textos de las reviews almacenados en MongoDB.

En Tableau, esta visualización se ha construido utilizando un gráfico de tipo texto. En este caso, la variable *palabra* se ha utilizado como elemento visual principal, mientras que la variable *frecuencia* se ha empleado para ajustar el tamaño de cada palabra. Además, se han aplicado ajustes de formato, como el uso de un fondo oscuro y colores contrastados, para mejorar la legibilidad.

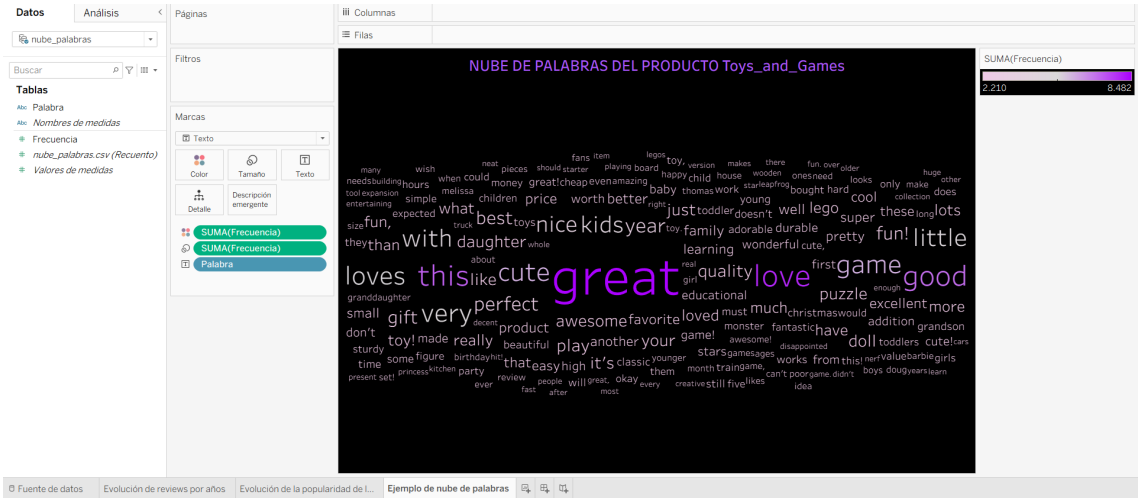


Figura 6.3: *Nube de palabras de la categoría Toys_and_Games*

Aunque Tableau no permite generar una distribución completamente aleatoria como la obtenida en Python con WordCloud, esta representación permite identificar fácilmente los términos más frecuentes. Destacan palabras como “great”, “love”, “good” o “this”, lo que sugiere una tendencia general hacia valoraciones positivas.

6.2 Diseño de un modelo de recomendación

A partir de los datos almacenados en la base de datos, es posible diseñar un sistema de recomendación orientado a sugerir artículos a usuarios en función de sus valoraciones previas. Con la información disponible se puede construir una matriz usuario-artículo, donde cada fila representa a un usuario y cada columna a un producto (asin) y los valores corresponden a las puntuaciones otorgadas.

Sobre esta estructura, se puede plantear un enfoque de filtrado colaborativo basado en usuarios, en la que las recomendaciones se generan a partir de la similitud entre usuarios. La idea principal consiste en identificar usuarios con patrones de valoración similares y utilizar sus preferencias para predecir qué artículos podrían resultar de interés a un usuario dado.

El proceso que seguiríamos sería:

1. Construcción de la matriz usuario-artículo a partir de las reviews almacenadas en MySQL.
2. Cálculo de similitudes entre usuarios (mediante correlación Pearson que ya ha sido utilizada)
3. Selección de vecinos más similares para cada usuario (mediante el modelo KNN)
4. Agregación de valoraciones de los vecinos para estimar la relevancia de artículos no consumidos
5. Generación de recomendaciones, priorizando aquellos artículos con mayor puntuación estimada.

Con este enfoque se puede aprovechar la información implícita en los patrones de comportamiento de los usuarios sin necesidad de conocer características explícitas de los productos.

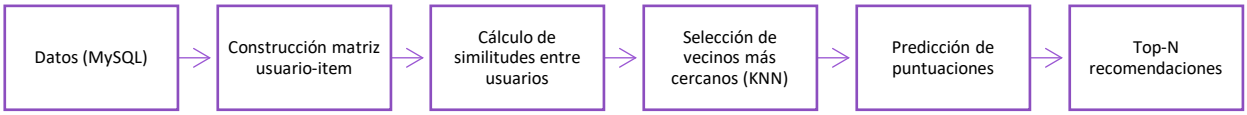


***Figura 6.4**: Diagrama del funcionamiento del sistema de recomendación*

6.3 Mecanismo de recomendación basado en popularidad

Como alternativa más sencilla y robusta, se ha implementado un mecanismo de recomendación basado en la popularidad de los artículos, condicionado por el historial del usuario.

Dado un identificador de usuario y una categoría de producto, el sistema:

1. Identifica los artículos ya consumidos por el usuario dentro de esa categoría
2. Calcula la popularidad de los artículos, definida como el número de reviews
3. Filtra los artículos no consumidos por el usuario
4. Ordena los resultados por popularidad
5. Devuelve los 10 artículos más relevantes

Este enfoque es más simple que el modelo de Machine Learning y presenta ventajas como que es más eficiente computacionalmente y funciona bien en situaciones que tienen mucha densidad de datos (el caso de los cinco datasets).

Además, permite obtener recomendaciones coherentes sin necesidad de un proceso de entrenamiento previo, lo que lo hace especialmente adecuado para integrarse directamente con la base de datos desarrollada.

Ejemplo de recomendaciones generadas

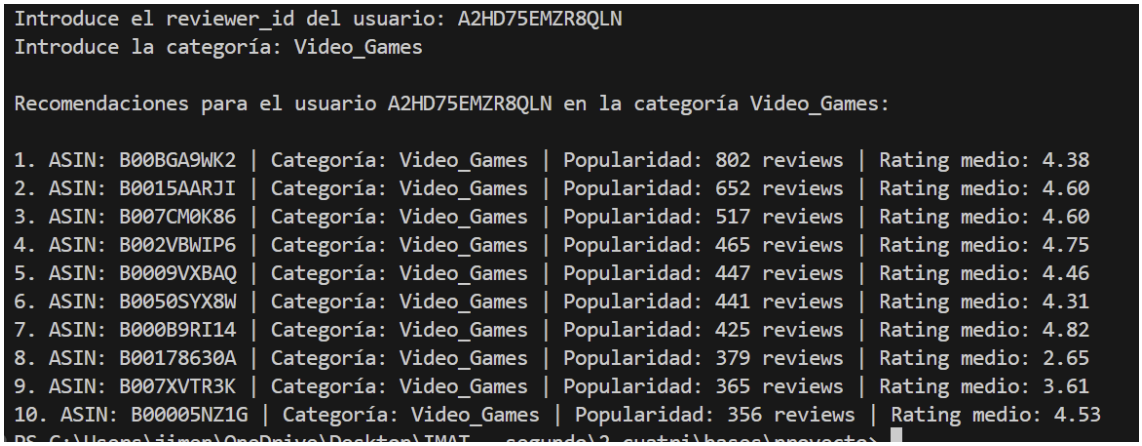


Figura 6.5: *Ejemplo de recomendaciones generadas*

Para validar el funcionamiento del mecanismo de recomendación implementado, se ha ejecutado el sistema proporcionando como entrada un identificador de usuario y una categoría concreta. En este caso, se ha utilizado el usuario A2HD75EMZR8QLN y la categoría Video_Games.

El sistema devuelve los 10 artículos más populares dentro de dicha categoría que el usuario no ha consumido previamente, junto con información relevante como el número de valoraciones (popularidad) y la puntuación media.

Los resultados obtenidos permiten extraer varias conclusiones relevantes:

En primer lugar, se observa que los artículos recomendados presentan un alto nivel de popularidad, medido a través del número de reviews. Esto indica que el sistema prioriza productos con una elevada interacción por parte de los usuarios, lo que suele correlacionarse con artículos ampliamente conocidos o demandados.

Además, la inclusión del rating medio aporta una segunda dimensión de calidad. En la mayoría de los casos, los artículos recomendados no solo son populares, sino que también presentan valoraciones elevadas (en torno a 4 o más), lo que refuerza la fiabilidad de las recomendaciones.

Este resultado confirma que el mecanismo implementado es capaz de generar recomendaciones relevantes de forma eficiente, utilizando únicamente la información almacenada en la base de datos, sin necesidad de entrenamiento previo de un modelo complejo.

6.4 Relación con Machine Learning

A partir de los datos almacenados en la base de datos, es posible diseñar sistemas de recomendación capaces de sugerir artículos a los usuarios en función de su comportamiento previo. En este proyecto se ha considerado la posibilidad de implementar un modelo basado en filtrado colaborativo, donde las recomendaciones se generan a partir de la similitud entre usuarios.

Sin embargo, aunque el enunciado propone como apartado opcional la implementación de un modelo de Machine Learning para la generación de recomendaciones, se decidió no incluir esta parte en la versión final del proyecto.

Durante el desarrollo se exploró la implementación de un modelo basado en K-Nearest Neighbors (KNN) para recomendación de artículos a partir del comportamiento de los usuarios. Sin embargo, tras realizar diferentes pruebas y evaluaciones sobre los datos disponibles, se observó que los resultados obtenidos no alcanzaban un nivel de calidad suficiente. En particular, el modelo presentaba una baja tasa de aciertos (3,33%) en las recomendaciones, debido principalmente a la escasa densidad de la matriz usuario-artículo y al reducido solapamiento entre usuarios en determinadas categorías.

Por este motivo, se centró el trabajo en las partes principales del proyecto, asegurando una implementación robusta y bien justificada de la arquitectura de bases de datos, la visualización de datos y el análisis mediante grafos.

6.5 Conclusión

Este apartado permite comprobar cómo una misma información puede representarse mediante diferentes herramientas, obteniendo resultados similares desde el punto de vista del análisis, pero con enfoques distintos en cuanto a la visualización. Mientras que Python ofrece un mayor control y personalización en el procesamiento de los datos y la generación de gráficas, Tableau facilita la construcción de visualizaciones de forma más rápida e intuitiva, permitiendo además una exploración más visual de la información.

La combinación de ambas herramientas aporta una visión más completa del conjunto de datos, ya que no solo permite generar representaciones gráficas, sino también comprender mejor los patrones y tendencias presentes en la información.

Por otro lado, aunque no se ha implementado finalmente un modelo de Machine Learning, el análisis realizado pone de manifiesto el potencial de los datos para desarrollar sistemas de recomendación en futuros trabajos.

7. Opcional: Mejora en visualización

Como ampliación de la segunda parte del proyecto, se ha desarrollado una versión más interactiva del sistema de visualización de datos empleando Streamlit. Mientras que en la parte obligatoria se pedía implementar un menú en Python, en este apartado se ha optado por construir un pequeño dashboard web que permite al usuario explorar las reviews de forma más intuitiva y cómoda.

La solución desarrollada mantiene la misma lógica funcional planteada en la segunda parte del proyecto, pero modifica la forma de interacción con el usuario. En esta versión se presenta una interfaz visual con dos desplegables principales: uno para seleccionar la categoría de productos y otro para elegir la gráfica que se desea visualizar. De este modo, el usuario puede cambiar de categoría o de tipo de gráfico de forma inmediata, sin necesidad de reiniciar la ejecución ni navegar por un menú repetitivo.

Para implementar esta mejora se ha utilizado la librería Streamlit, que permite construir interfaces interactivas en Python de forma sencilla. Además, se siguen utilizando las mismas tecnologías del resto del proyecto para acceder a la información: PyMySQL para la conexión con la base de datos relacional en MySQL, PyMongo para el acceso a MongoDB, Matplotlib para la generación de las gráficas y WordCloud para la nube de palabras. Por tanto, esta parte no sustituye la lógica de acceso a los datos ya desarrollada, sino que la reutiliza dentro de una capa de presentación más visual e interactiva. Como indica el enunciado del opcional, los paquetes adicionales necesarios deben reflejarse en un fichero requirements.txt, por lo que en dicho archivo se incluyen las dependencias empleadas en esta mejora.

La estructura del código se ha organizado en funciones, destacando especialmente la función obtener_categorias(), que consulta dinámicamente las categorías presentes en la tabla *productos* y permite rellenar automáticamente el desplegable de selección. De esta forma, la interfaz no depende de valores definidos manualmente, sino que se adapta al contenido real de la base de datos.

Las visualizaciones implementadas recogen las mismas ideas desarrolladas en la segunda parte del proyecto, por lo que no se detallan nuevamente en este apartado.

En cuanto a las ventajas de esta mejora, la principal aportación es la interactividad. Además, se consigue una presentación más clara de los resultados, especialmente al utilizar un formato de página amplia y mostrar las gráficas integradas en una aplicación web ligera. Todo ello hace que la exploración de los datos sea más natural y cercana a un dashboard real.

Por último, para ejecutar esta parte opcional es necesario disponer de las dependencias adicionales utilizadas en la interfaz. Estas se han recogido en el fichero requirements.txt, incluyendo al menos *streamlit*, *pymysql*, *pymongo*, *matplotlib* y *wordcloud*, además de las librerías ya empleadas en el resto del proyecto.

8. Conclusión

A lo largo de este proyecto se ha abordado de forma completa el ciclo de vida de los datos, desde su modelado y carga inicial hasta su explotación, análisis y extensión mediante distintas tecnologías. Partiendo de los datasets de reviews de Amazon, se ha diseñado una arquitectura híbrida capaz de gestionar tanto información estructurada como no estructurada, combinando de manera coherente el uso de MySQL y MongoDB. Esta decisión ha permitido aprovechar las ventajas de cada modelo de datos, logrando un sistema eficiente, flexible y adaptado a la naturaleza de la información.

En una segunda fase, se ha desarrollado una aplicación en Python que permite acceder y visualizar los datos, transformando grandes volúmenes de información en representaciones gráficas comprensibles. Este paso ha sido clave para pasar de un almacenamiento pasivo a una explotación activa de los datos, facilitando el análisis del comportamiento de los usuarios, la popularidad de los productos y la evolución temporal de las reviews.

Posteriormente, la incorporación de Neo4J ha supuesto un cambio de perspectiva en el análisis, permitiendo modelar explícitamente las relaciones entre usuarios, productos y categorías. A través de distintos grafos, se ha podido estudiar la similitud entre usuarios, identificar patrones de consumo y representar interacciones complejas que resultan difíciles de capturar en modelos relacionales tradicionales. Este enfoque ha demostrado el valor de los grafos como herramienta complementaria para el análisis de datos altamente interconectados.

Además, el sistema se ha diseñado con capacidad de evolución, implementando un mecanismo de carga incremental que permite incorporar nuevos datasets sin reconstruir la base de datos desde cero. Este aspecto resulta especialmente relevante en escenarios reales, donde los datos crecen de forma continua y es necesario garantizar la consistencia y la robustez del sistema.

Finalmente, se ha explorado la relación entre los datos almacenados y posibles aplicaciones de Machine Learning, planteando y desarrollando un sistema de recomendación basado tanto en similitud entre usuarios como en popularidad de los artículos. Este último paso pone de manifiesto cómo una correcta gestión y estructuración de los datos constituye la base necesaria para desarrollar sistemas inteligentes capaces de generar valor a partir de la información.

En conjunto, este proyecto demuestra la importancia de integrar diferentes tecnologías de bases de datos y herramientas de análisis para abordar problemas reales de datos. Más allá de cumplir los requisitos planteados, se ha construido una solución completa, escalable y extensible, que no solo permite almacenar y consultar información, sino también extraer conocimiento relevante y sentar las bases para futuras aplicaciones más avanzadas.